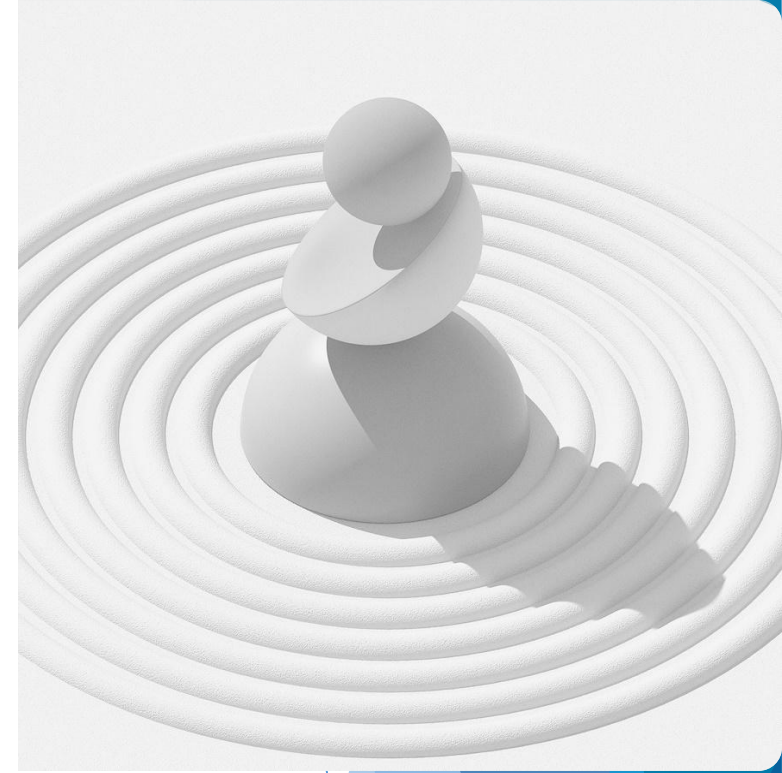


Presented by Nirvana Popescu

Lesson 4: Requirements and Traceability in Safety-Critical Software Engineering

Outline

- ▶ Why are requirements important?
- ▶ What are requirements?
- ▶ Good requirements and how to build them
- ▶ Requirements engineering process
- ▶ Safety requirements
- ▶ Traceability
- ▶ Traceability examples
- ▶ Why it matters?



Why are requirements important?

- ▶ Requirements define the problem before a solution can be designed.
- ▶ They describe what the system must achieve, while design and implementation determine how to achieve it.



requirements define the problem space, while design and implementation explore the solution space.

- ▶ Writing requirements makes knowledge explicit by revealing assumptions, gaps, and inconsistencies.
- ▶ Requirements help engineers reason about the system's properties, constraints, and objectives before implementation.
- ▶ Explicit requirements enable analysis, verification, validation, and traceability throughout the development lifecycle.
- ▶ What cannot be stated precisely cannot be analyzed, verified, or justified.



What are requirements?

A **requirement** is a documented statement describing:

- ▶ what a system shall do,
- ▶ how well it shall perform,
- ▶ constraints under which it must operate.

Types of Requirements

- ▶ **Business Requirements**
 - ▶ Project goals
 - ▶ Regulatory objectives
- ▶ **Stakeholder Requirements**
 - ▶ User expectations
 - ▶ Operational needs
- ▶ **System Requirements**
 - ▶ Functional requirements
 - ▶ Non-functional requirements
 - ▶ Safety requirements

System Requirements Examples

Functional:

The braking system shall apply emergency braking within 100 ms after receipt of a valid emergency braking request.

Non-functional:

The braking system shall have an availability of at least 99.99% during vehicle operation.

Safety:

The braking system shall never command acceleration during emergency braking.



Characteristics of Good Requirements

- ▶ A good requirement should be:
 - ▶ **Correct** - reflects stakeholder needs.
 - ▶ **Complete** - contains all necessary information.
 - ▶ **Consistent** - does not contradict other requirements.
 - ▶ **Unambiguous** - interpreted in only one way.
 - ▶ **Verifiable** - can be inspected and tested.
 - ▶ **Feasible** - technically achievable.
 - ▶ **Traceable** - linked to its source and implementation.
 - ▶ **Modifiable** - easy to update.
- ▶ **Poor Requirement**
 - ▶ The system should respond quickly.
- ▶ **Improved Requirement**
 - ▶ The system shall display the warning message within 2 seconds after detecting a critical fault.



How to build good requirements?

- ▶ Poorly written requirements create ambiguity, inconsistencies, and incomplete specifications, making systems difficult to design, verify, and test.
- ▶ A structured method, such as EARS, helps produce clear, consistent, and verifiable requirements, reducing development costs and improving safety.
- ▶ EARS - Easy Approach to Requirements Syntax
 - ▶ EARS is a lightweight requirements-writing method that:
 - ▶ Provides a simple, standardized syntax for writing requirements.
 - ▶ Improves clarity, consistency, and stakeholder communication.
 - ▶ Facilitates verification, traceability, and certification in safety-critical systems.

Please consult the details offered by Ansys Innovation Space courses:

<https://innovationspace.ansys.com/courses/courses/development-of-adas-functions-with-ansys-scade/lessons/lesson-1-ears-method-to-write-clear-system-requirements/>



How Are Requirements Built?

Requirements Engineering Process

- ▶ **Elicitation**
 - ▶ Interviews
 - ▶ Workshops
 - ▶ Observation
 - ▶ Standards and regulations
- ▶ **Analysis**
 - ▶ Resolve conflicts
 - ▶ Prioritize requirements
 - ▶ Assess risks
- ▶ **Specification**
 - ▶ Produce Software Requirements Specification (SRS)
- ▶ **Validation**
 - ▶ Reviews
 - ▶ Inspections
 - ▶ Prototyping
 - ▶ Testing
- ▶ **Management**
 - ▶ Version control
 - ▶ Change management
 - ▶ Traceability

Safety Requirements

- ▶ Safety requirements originate from:

- ▶ Hazard Analysis
- ▶ Risk Assessment
- ▶ Regulatory standards
- ▶ Safety goals

- ▶ Example:

- ▶ Hazard:
Vehicle fails to stop.



- ▶ Safety Requirement:
The braking controller shall detect sensor failures within 10 ms and switch to a safe operation mode.

Example process:



What Is Requirements Traceability?

► Definition

- Requirements traceability is the ability to follow the life of a requirement from its origin through development, verification, deployment, and maintenance.

► Traceability answers questions such as:

- Where did this requirement come from?
- Which design component implements it?
- Which code modules satisfy it?
- Which tests verify it?
- What changes if this requirement is modified?

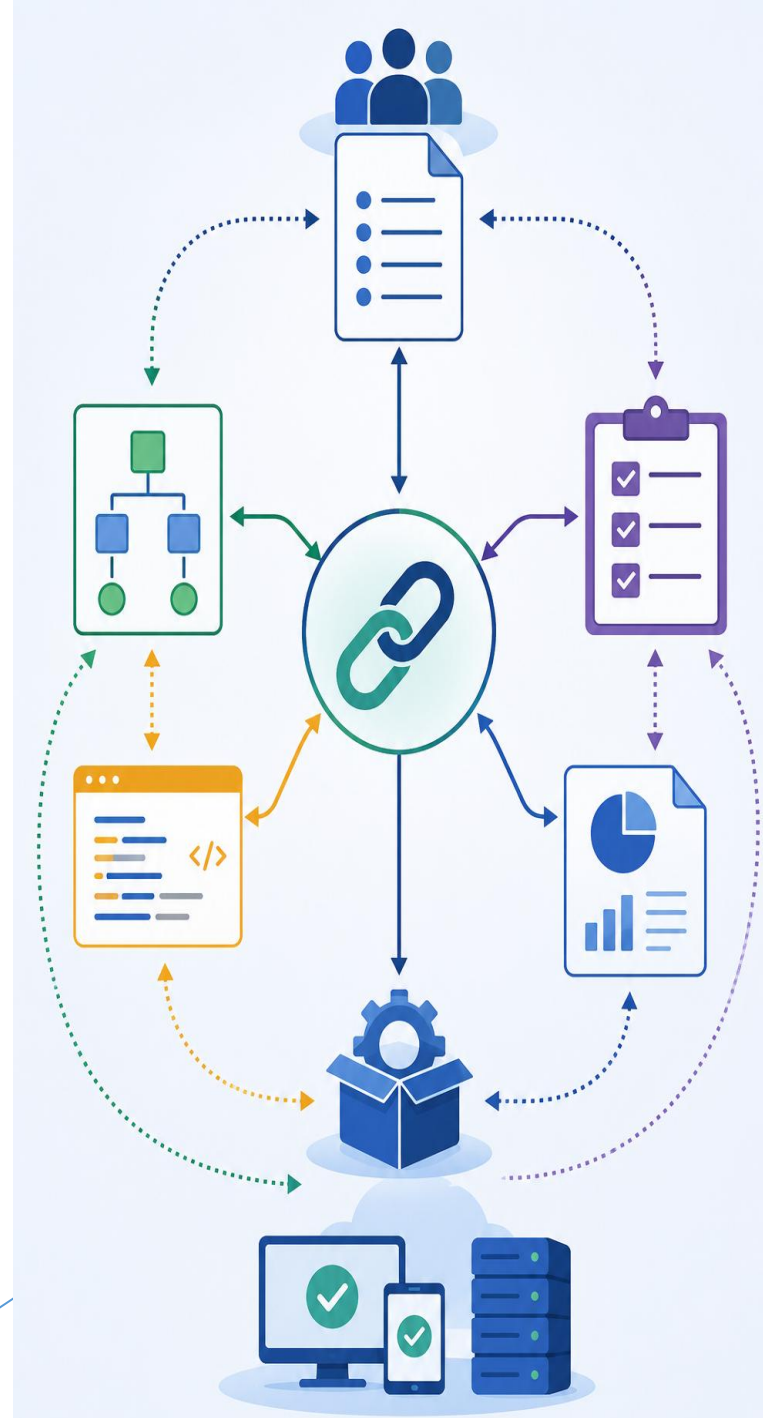
► Types

► Forward Traceability

Requirement → Design → Code → Test

► Backward Traceability

Test → Code → Design → Requirement → Stakeholder Need



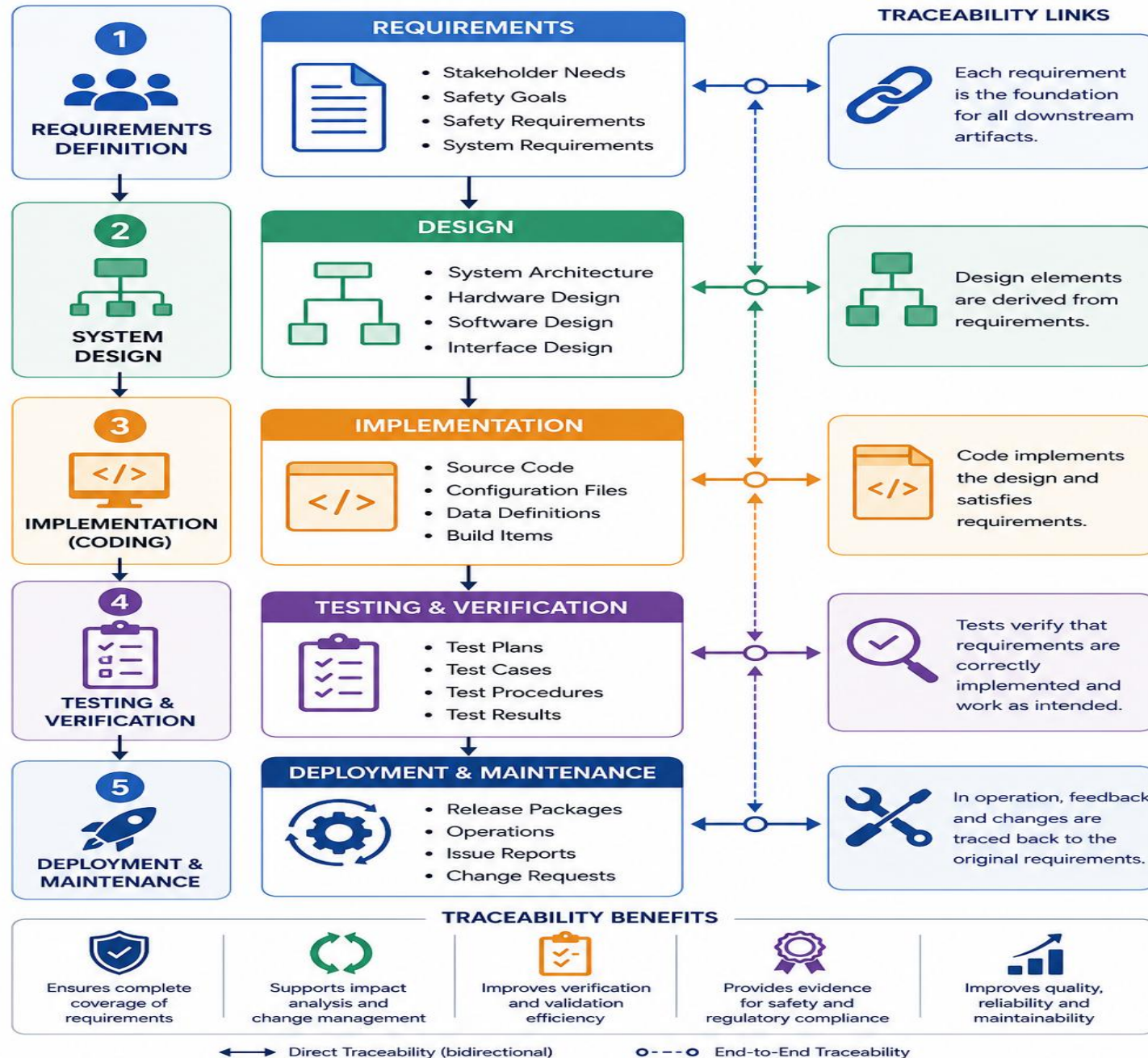
Traceability Matrix Example

Requirement	Design Component	Code/line Module	Test Case
R-001 Emergency braking within 100 ms	Brake Controller	BrakeControl.cpp	TC-001
R-002 Detect sensor failure	Diagnostics Module	SensorMonitor.cpp	TC-014
R-003 Activate warning light	HMI Controller	WarningDisplay.cpp	TC-020

► Benefits

- Easier impact analysis
- Demonstrates compliance
- Simplifies certification
- Improves maintenance
- Ensures complete testing

TRACEABILITY THROUGHOUT THE DEVELOPMENT LIFECYCLE



Why Traceability Matters in Safety-Critical Systems

- ▶ Without traceability:
 - ▶ Missing requirements may never be implemented.
 - ▶ Tests may not cover all safety requirements.
 - ▶ Changes can introduce hidden hazards.
 - ▶ Certification becomes difficult.
 - ▶ Fault investigation is much slower.
- ▶ With traceability:
 - ▶ Every safety requirement is implemented.
 - ▶ Every implementation is justified.
 - ▶ Every requirement is verified.
 - ▶ Every change can be assessed for impact.



Summary

- ▶ Requirements define what a safety-critical system shall do.
- ▶ Good requirements are clear, complete, verifiable, and traceable.
- ▶ Requirements engineering includes elicitation, analysis, specification, validation, and management.
- ▶ Traceability links requirements to design, implementation, testing, and certification.
- ▶ Traceability is essential for change management, verification, and demonstrating compliance in safety-critical software.

